UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Experiment No-3

Student Name: Somnath Pal
UID: 24MCA20207
Branch: MCA
Section/Group: 24MCA-1(B)
Semester: 3rd
Date of Performance: 09/09/2025
Subject Name: Machine Learning Lab
Subject Code: 24CAP-702

Aim: a) Write a program for Support Vector Machine using python for classification and analyze its performance using the Iris dataset.

- Define any problem which can be solved using SVM
- Specify datasets for your problem
- Execute and discuss the outcomes of the experiment

b) Write a program based on logistic regression and make the sigmoid graph using sigmoid function.

Objective:
We aim to classify Iris flower species into two categories (Setosa and Versicolor) based on their sepal length and sepal width. SVM will be used to find the optimal decision boundary between the two classes.

Algorithm:
Step 1: Import Libraries
• Import the required libraries such as NumPy, Matplotlib, Seaborn, and scikit-learn.

Step 2: Load Dataset
• Load the Iris dataset from sklearn.datasets.
• Select two features (Sepal Length, Sepal Width) for classification.

Step 3: Data Preprocessing
• Select only two classes: Setosa and Versicolor.
• Split the dataset into training and testing sets (70%-30%).

Step 4: Train the Model
• Use Support Vector Classifier (SVC) from scikit-learn.

• Set the kernel to linear to draw a straight decision boundary.
• Fit the model using the training data.

Step 5: Test the Model
• Predict outcomes for test data using the trained model.

Step 6: Evaluate Performance
• Calculate accuracy score.
• Generate the classification report.
• Visualize the confusion matrix.
• Draw the decision boundary for classification.

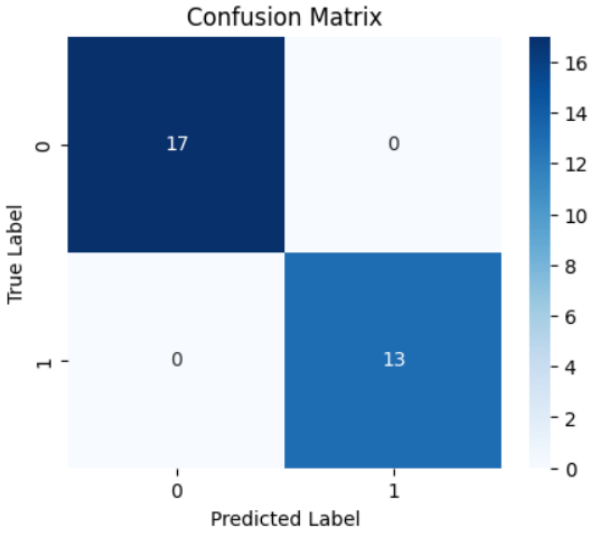
Step 7: Display Results
• Print evaluation metrics.
• Show graphical visualization.

Code:

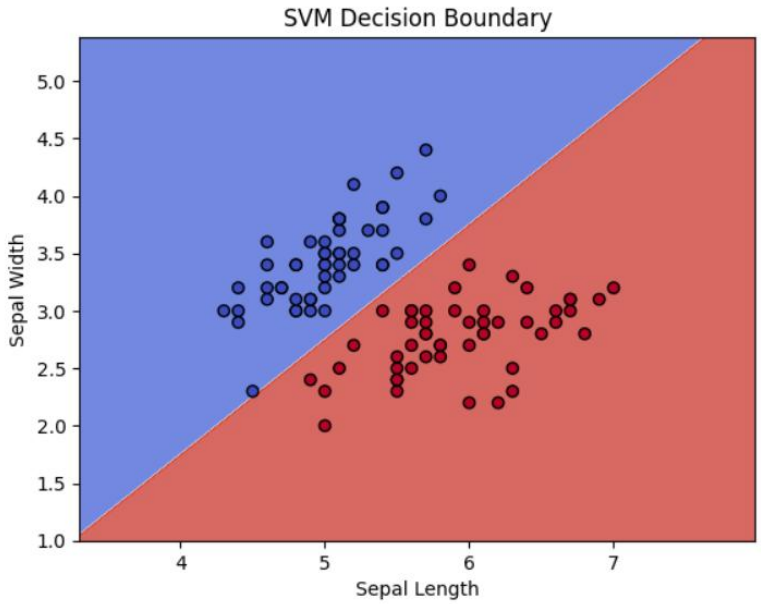
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = datasets.load_iris()
X = iris.data[:, :2] # Taking only sepal length & sepal width for visualization
y = iris.target
X = X[y != 2]
y = y[y != 2]

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=42)

model = SVC(kernel='linear') # Linear kernel
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
plt.figure(figsize=(5, 4))

sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()

h = .02
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
 np.arange(y_min, y_max, h))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, edgecolors='k')
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("SVM Decision Boundary")
plt.show()
```

Output:



```
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        17
           1       1.00      1.00      1.00        13

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```



Learning Outcomes:

1. Understood how to load and preprocess datasets in scikit-learn.
2. Learned the difference between data (features) and target (labels).
3. Learned how an SVM with a linear kernel separates two classes using a hyperplane.
4. Used accuracy, classification report (precision, recall, F1-score), and confusion matrix.
5. Understood what the SVM "decision boundary" looks like in 2D.